

VNR-QCKIT — Luồng hoạt động, Input/Output từng Skill

Tài liệu này mô tả tổng quan kiến trúc, giai đoạn, và chi tiết Input/Output của từng skill trong plugin **vnr-qckit**. Cập nhật: 2026-06-11 (rev2)

Tổng quan kiến trúc

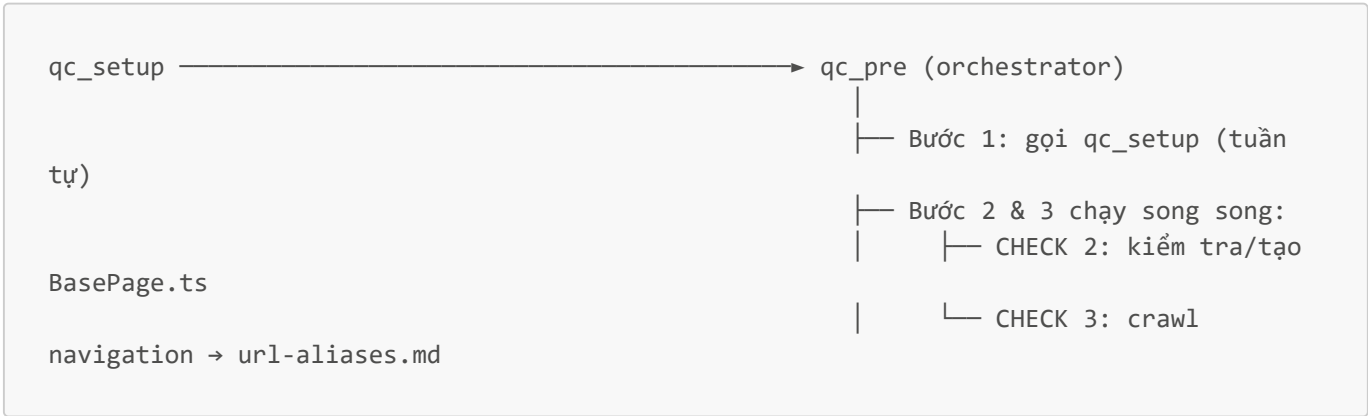
CÀI ĐẶT & KHỞI TẠO CASE	KHÁM PHÁ ỨNG DỤNG	SINH TEST
qc_setup qc_generate ↳ cài Playwright + Chromium sinh testcase.md (Mục 4) ↳ tạo playwright.config.ts ↳ sinh datafake.json ↳ tạo global-setup.ts ↳ sinh knowledge/*.md ↳ cập nhật _IMPACT_INDEX.json qc_pre (orchestrator) ↳ gọi qc_setup & PHÂN TÍCH ↳ gọi qc_basepage	qc_detect_component ↳ crawl DOM (playwright-mcp) ↳ sinh component_{alias}.md (.specify/memory/components/) qc_component_rule ↳ đọc component_*.md ↳ sinh/cập nhật component-rule.md	↳
↳ crawl navigation qc_auto_test ↳ sinh url-aliases.md lọc TC Auto chưa PASSED (Mục 4) thực thi mcp + playwright-cli qc_basepage - ghi traces/ + screenshots/ ↳ tạo BasePage.ts cập nhật Mục 4 testcase.md ghi HISTORY.md	qc_user_flow ↳ crawl từng màn hình (playwright-mcp) ↳ sinh ASCII flow diagram 5 tầng ↳ sinh userflow-{group}.md ↳ cập nhật flow-index.md	↳ ↳ ↳ ↳
MAPPING PAGE OBJECT qc_triage		↳
đọc HISTORY.md mới nhất qc_map_flow phân loại PASS/APP_BUG/INFRA ↳ đọc component-rule.md + userflow sinh triage-{PBI_ID}-{DATE}.md ↳ sinh {AliasName}Locator.ts (pages/{group}/) ↳ sinh {AliasName}Page.ts (pages/{group}/{alias}/) ↳ sinh locator-map.md qc_url_page_map		↳ ↳ ↳

```
└─ quét *Page.ts + *Locator.ts
└─ sinh url-page-map.md

qc_sub-system-map
└─ đọc url-page-map.md
└─ sinh sub-system-map.json
```

Giai đoạn 1 — Cài đặt & Khởi tạo Project

Chạy **một lần** khi setup project mới hoặc reset cấu hình.



Skill: **qc_setup** (Internal)

Được gọi bởi **qc_pre**. Có thể chạy độc lập.

Mục đích	Cài đặt Playwright + Chromium, tạo/kiểm tra file cấu hình
Trigger	Gọi bởi qc_pre , hoặc thủ công khi reset cấu hình

INPUT:

File/Thư mục	Bắt buộc	Ghi chú
.specify/tests/.env	✓	Phải tồn tại trước
{PLUGIN_DIR}/tools/pw-chromium/chrome-win	✓	Chromium bundled sẵn trong plugin

OUTPUT:

File	Trạng thái	Ghi chú
.specify/tests/ (npm package)	Tạo/cập nhật	@playwright/test được install
%LOCALAPPDATA%\pw-chromium\chrome-win\	Copy	Chromium cố định, dùng làm executablePath
.specify/tests/tsconfig.json	Tạo nếu thiếu	ES2020 , commonjs
.specify/tests/playwright.config.ts	Tạo/sửa nếu thiếu chuẩn	headless, readEnvFile, REPORT_DIR
.specify/tests/tests/ (thư mục)	Tạo nếu thiếu	testDir cho Playwright

File	Trạng thái	Ghi chú
<code>.specify/tests/global-setup.ts</code>	Tạo nếu thiếu	session login

Luồng xử lý (6 CHECK tuần tự):

```

CHECK 0 → Install Playwright + copy Chromium + tạo tests/ + tsconfig.json
CHECK 1 → playwright.config.ts tồn tại? MISSING → tạo mới → skip 2/3/4
CHECK 2 → readEnvFile có trong config? MISSING → chèn hàm readEnvFile
CHECK 3 → headless: true? MISSING → sửa thành true
CHECK 4 → REPORT_DIR? MISSING → thêm biến reportDir
CHECK 5 → global-setup.ts tồn tại? MISSING → hỏi user chọn cách tạo

```

Skill: `qc_basepage` (Internal)

Được gọi bởi `qc_pre`. Có thể chạy độc lập.

Mục đích	Tạo hoặc cập nhật <code>BasePage.ts</code> — lớp cha chứa common methods cho mọi Page Object
Trigger	Gọi bởi <code>qc_pre</code> khi thiếu <code>BasePage</code> , hoặc thủ công khi cần cập nhật

INPUT:

File	Bắt buộc	Ghi chú
<code>.specify/tests/pages/BasePage.ts</code>	—	Đọc nếu đã tồn tại để tránh ghi đè
<code>.specify/tests/.env</code> + <code>.env.{ENV}</code>	✓	Lấy APP_URL để verify

OUTPUT:

File	Trạng thái	Ghi chú
<code>.specify/tests/pages/BasePage.ts</code>	Tạo mới / giữ nguyên	Template hardcoded trong SKILL.md
<code>.specify/tests/tmp/_basepage_verify.spec.ts</code>	Tạm, xóa sau	Test tạm để verify methods

Methods trong BasePage.ts:

```

navigate(url)           - goto + waitUntil domcontentloaded
handleConfirmDialog()   - click confirm nếu xuất hiện
waitForToast()          - chờ toast/notification
verifyUrl()              - assert URL
inputTextbox()           - nhập text vào textbox
verifyTextbox()          - verify value textbox
inputCombobox()          - nhập + chọn từ dropdown jQuery autocomplete
inputCheckbox()          - tích / bỏ tích checkbox
verifyCheckbox()         - verify trạng thái checkbox
inputTable()             - nhập dữ liệu vào bảng (TableRow[])

```

Luồng xử lý:

```
Bước 1 → Kiểm tra file đã tồn tại?  
Bước 2 → Tạo mới từ template (nếu thiếu)  
Bước 3 → Tạo test tạm → chạy Playwright verify từng method  
Bước 4 → Sửa selector/timeout nếu method fail (KHÔNG sửa signature/tên)  
Bước 5 → Xóa test tạm
```

Skill: **qc_pre** (Orchestrator)

Mục đích Khởi tạo toàn bộ project QC: setup Playwright, BasePage, crawl URL, sinh url-aliases.md

Trigger Chạy thủ công 1 lần khi init project hoặc refresh URL aliases

INPUT:

File	Bắt buộc	Ghi chú
<code>.specify/tests/.env</code>	✓	Chứa <code>ENV=local</code> (hoặc tên env)
<code>.specify/tests/.env.{ENV}</code>	✓	Chứa <code>APP_URL=http://...</code>
<code>.specify/tests/url-aliases.md</code>	—	Đọc nếu đã tồn tại để giữ alias cũ

OUTPUT:

File	Trạng thái	Ghi chú
(via <code>qc_setup</code>)	—	Playwright config, Chromium, global-setup
<code>.specify/tests/pages/BasePage.ts</code>	(via <code>qc_basepage</code>)	Tạo nếu thiếu
<code>.specify/tests/url-aliases.md</code>	Tạo mới / cập nhật	Bảng alias → path từ crawl navigation

Format url-aliases.md:

```
# URL Aliases  
> ENV: local | APP_URL: http://...  
  
## Tên nhóm (group-code)  
| Alias | Path (Local) | Path (dev) | Mô tả |  
| aw_home | /agentwork/home | /home | Trang chủ |
```

Quy tắc sinh alias:

- `/agentwork/ai-employee` → prefix `aw_` + snake_case → `aw_ai_employee`
- Query param `?tab=log` → suffix → `aw_settings_log`
- Alias cũ: **không đổi tên**, chỉ cập nhật path nếu route đổi

Luồng xử lý (3 bước):

Bước 1 → Gọi qc_setup (tuần tự – Playwright config + Chromium + global-setup)

Bước 2 → CHECK 2 & CHECK 3 chạy song song:

CHECK 2 → Kiểm tra BasePage.ts → gọi qc_basepage nếu thiếu

CHECK 3 → Đọc APP_URL từ .env → Crawl navigation → Ghi/cập nhật url-aliases.md

Giai đoạn 2 — Khám phá Ứng dụng

Crawl DOM và user flow để hiểu cấu trúc màn hình, phát hiện component, sinh selector rules.

qc_detect_component

- ↳ crawl DOM qua playwright-mcp
- .specify/memory/components/
 - ↳ phát hiện rule-component
 - ↳ phát hiện unknown input
 - ↳ sinh component_{alias}.md (.specify/memory/components/)

qc_component_rule

- ↳ đọc component_*.md từ
 - ↳ tra DOM HTML mẫu
 - ↳ sinh/cập nhật component-rule.md

qc_user_flow

- ↳ crawl action từng màn hình (playwright-mcp)
 - ↳ vẽ ASCII flow 5 tầng
 - ↳ sinh userflow-{group}.md
 - ↳ cập nhật flow-index.md

Skill: qc_detect_component

Mục đích	Crawl DOM từng màn hình qua playwright-mcp, phát hiện component theo component-rule.md và unknown inputs, sinh file mô tả component theo screen
----------	---

Arguments	[group-code] / [alias1 alias2] / --all
-----------	--

INPUT:

File	Bắt buộc	Ghi chú
.specify/tests/url-aliases.md	✓	Danh sách màn hình cần crawl
.specify/rules/component-rule.md	✓	Định nghĩa selector & container rule
.specify/tests/user.json	✓	Session login; tự re-login nếu hết hạn
.specify/tests/.env + .env.{ENV}	✓	Lấy APP_URL, AUTH_USERNAME, AUTH_PASSWORD

OUTPUT:

File	Trạng thái	Ghi chú
.specify/memory/components/component_{alias}.md	Tạo mới / ghi đè	Một file per screen; chứa danh sách component + fields

Format component_{alias}.md:

```
# Component List – {alias}
> Screen    : {mô tả từ url-aliases.md}
> URL       : {path}
> Sinh bởi  : /qc_detect_component – {ISO_DATE}
> Rule      : .specify/rules/component-rule.md
> Tổng     : {n} rule-components, {m} unknown-inputs

---

## [RULE COMPONENTS] – Khớp với component-rule.md

### k-textbox ({count} fields)
| # | Label | Name | Placeholder | Required | Context |
| 1 | Mã Loại | Code | Nhập mã | ✓ | main-form |

### k-button ({count} buttons)
| # | Label | Context |
| 1 | Lưu | main-form |

---

## [UNKNOWN INPUTS] – Element nhập liệu chưa có trong component-rule.md
| # | Label | Name | Type | CSS Classes | Placeholder | Required | Context | HTML Snippet |
```

Luồng xử lý:

BƯỚC 0 → Đọc env, kiểm tra url-aliases.md + component-rule.md, kiểm tra session qua playwright-mcp

Nếu session hết hạn → tự đăng nhập lại (navigate → type → click → wait)

BƯỚC 1 → Parse url-aliases.md, lọc alias theo argument; nhóm alias theo màn hình gốc (bỏ tab con)

BƯỚC 2 → Parse component-rule.md → danh sách rule-component (container, input, label selectors)

BƯỚC 3 → Crawl DOM từng màn hình qua playwright-mcp:

- 3.1 browser_navigate → networkidle
- 3.2 browser_snapshot (tuỳ chọn debug)
- 3.3 browser_evaluate – detect rule-components (1 lần IIFE tổng hợp tất cả rules)
- 3.4 browser_evaluate – detect unknown inputs (chưa bị rule-component bao phủ)

BƯỚC 4 → Sinh .specify/memory/components/component_{alias}.md (ghi đè nếu tồn tại)

BƯỚC 5 → Báo cáo tổng hợp

Giới hạn: Mỗi màn hình tối đa 30 giây. Session hết hạn → dừng toàn bộ.

Skill: `qc_component_rule`

⚠ Đây là skill thay thế cho `qc_init_component` cũ. Khác biệt chính: đọc từ `.specify/memory/components/` (output của `qc_detect_component`), chỉ xử lý NEW/INCOMPLETE, hỗ trợ tham số component cụ thể.

Mục đích	Tổng hợp component types từ <code>component_*.md</code> , tra DOM HTML mẫu, ghi định nghĩa selector vào <code>component-rule.md</code>
Arguments	(không có) → xử lý tất cả / <code><component-name></code> → force update component cụ thể
Trigger	Chạy sau <code>qc_detect_component</code> khi phát hiện component mới

INPUT:

File	Bắt buộc	Ghi chú
<code>.specify/memory/components/component_*.md</code>	✓	Ít nhất 1 file; thiếu → DỪNG
<code>{PLUGIN_DIR}/detected_components/components.md</code>	—	DOM HTML reference cho Angular components
<code>{PLUGIN_DIR}/components_core.md</code>	—	DOM HTML reference cho Kendo/legacy
<code>{PLUGIN_DIR}/templates/template-component-rule.md</code>	—	Template format output
<code>.specify/rules/component-rule.md</code>	—	File đích; đọc nếu tồn tại để tránh ghi đè entry cũ

OUTPUT:

File	Trạng thái	Ghi chú
<code>.specify/rules/component-rule.md</code>	Tạo mới / append	Định nghĩa selector cho từng component type

Trạng thái xử lý từng component:

Trạng thái	Hành động
Chưa có entry <code>## {COMPONENT_TYPE}</code>	→ NEW — thêm mới
Đã có entry nhưng có <code>[TODO: cần clarify selector]</code>	→ INCOMPLETE — hỏi user có muốn cập nhật
Đã có entry đầy đủ	→ SKIP — không xử lý

Luồng xử lý (7 Bước):

```
BƯỚC 0 → Quét .specify/memory/components/, phân loại files
BƯỚC 1 → Đọc song song: component-rule.md (file đích), templates, DOM reference
BƯỚC 2 → Tổng hợp unique component types từ tất cả component_*.md
           Lọc NEW + INCOMPLETE (so với component-rule.md hiện tại)
BƯỚC 3 → Phân tích DOM cho từng component NEW/INCOMPLETE:
           3.1 Tìm DOM HTML mẫu (ưu tiên components.md → components_core.md)
           3.2 Trích xuất container, label, input selectors
           3.3 Bổ sung selector động (placeholder, count suffix, validate area...)
BƯỚC 4 → Hiển thị preview danh sách thay đổi → xác nhận trước khi ghi
BƯỚC 5 → Ghi/append vào .specify/rules/component-rule.md
```

BƯỚC 6 → Cập nhật metadata header trong component-rule.md
BƯỚC 7 → Báo cáo tổng hợp (CREATED / UPDATED / SKIPPED / TODO)

Skill: qc_user_flow

Mục đích	Crawl luồng thực tế qua playwright-mcp, vẽ ASCII flow diagram 5 tầng (Group→Alias→Feature→Screen→Action), sinh userflow-{group}.md, cập nhật flow-index.md
Arguments	[alias1 alias2] / [GROUP-CODE] / --all

INPUT:

File	Bắt buộc	Ghi chú
.specify/tests/url-aliases.md	✓	Danh sách màn hình cần crawl
.specify/tests/.env + .env.{ENV}	✓	APP_URL, AUTH_USERNAME, AUTH_PASSWORD
.specify/tests/user.json	✓	Session login
.specify/memory/flow-index.md	—	Tạo mới nếu chưa có; cập nhật incremental nếu đã có

OUTPUT:

File	Trạng thái	Ghi chú
.specify/memory/userflow-{group}.md	Tạo mới / cập nhật	Sơ đồ ASCII 5 tầng + chi tiết từng màn hình
.specify/memory/flow-index.md	Tạo mới / cập nhật incremental	Bảng trạng thái tất cả alias đã crawl

Quy tắc đặt tên file output:

Input	File output
--all hoặc không có argument	userflow-all.md
1 group code (ví dụ ATT)	userflow-att.md
Nhiều group code	userflow-att-hre.md
1 alias lẻ	userflow-cat_leave_day_type.md

Format sơ đồ ASCII (phân cấp 5 tầng):

```
GROUP: ATT — Cham cong
|
|— [att_leave_day] DS Ngày nghỉ
|   URL: /Att_LeaveDay/Index | Loại: List screen
|   |
|   |— FEATURE: Tạo mới
|   |   SCREEN: (Dialog: Tạo ngày nghỉ)
|   |       Nhập: Loại ngày nghỉ*, Từ ngày*, Đến ngày*, Lý do
```



```

--[Lưu]-->  ✓ toast: "Lưu thành công"    --> [att_leave_day]
--[Hủy]-->  [att_leave_day]

FEATURE: Xóa
SCREEN: (Confirm: Xác nhận xóa?)
--[Đồng ý]--> ✓ toast: "Xóa thành công"    --> [att_leave_day]

```

Format flow-index.md:

```

| Group | Alias | Mô tả | URL (local) | Trạng thái | Thời gian crawl | File userflow |
| ATT | att_leave_day | DS Ngày nghỉ | /Att_LeaveDay/Index | ✓ OK | 2026-06-11 10:30
| userflow-att.md |

```

Mã trạng thái flow-index: ✓ OK / ✗ FORBIDDEN / ✗ NOT_FOUND / 🕒 TIMEOUT / ⚠ NO_DATA / 🔄 CHƯA CRAWL

Luồng xử lý (8 Bước):

BƯỚC 0 → Kiểm tra env, url-aliases.md, session; tạo flow-index.md nếu thiếu
 BƯỚC 1 → Parse url-aliases.md, resolve alias theo argument; suy luận Feature từ prefix alias
 BƯỚC 2 → Crawl từng màn hình qua playwright-mcp (tối đa 60 giây/màn hình):
 2.1 browser_navigate
 2.2 browser_snapshot → nhận diện loại màn hình
 2.3 Crawl toolbar actions (safe buttons: Thêm mới, Sửa, Tìm kiếm)
 2.4 browser_click → mở dialog/panel → crawl fields → Escape đóng
 2.5 Crawl tabs (màn hình Tab screen)
 2.6 Crawl search/filter
 BƯỚC 3 → Suy luận happy path; xây navigation graph; map field_type → BasePage method
 BƯỚC 4 → Vẽ sơ đồ ASCII 5 tầng (GROUP→ALIAS→FEATURE→SCREEN→ACTION)
 BƯỚC 5 → Ghi userflow-{group}.md
 BƯỚC 6 → Cập nhật flow-index.md (incremental – không reset)
 BƯỚC 7 → Cập nhật url-aliases.md nếu phát hiện bất thường (chỉ thêm comment)
 BƯỚC 8 → Sinh Playwright spec tái sử dụng qua /playwright-cli (cho alias ✓ OK)

Giai đoạn 3 — Mapping Page Object

Tạo Page Object TypeScript và mapping alias → Page Object.

qc_map_flow system-map	qc_url_page_map	qc_sub-
↳ đọc component-rule.md + userflow	↳ quét *Page.ts + *Locator.ts	↳ đọc
url-page-map.md		
↳ tính locator strategy	↳ khớp alias → Page class	↳ sinh
sub-system-map.json		
↳ sinh {AliasName}Locator.ts	↳ sinh url-page-map.md	
↳ sinh {AliasName}Page.ts		
↳ sinh locator-map.md		

Skill: qc_map_flow

Mục đích	Sinh 2 file TypeScript tách biệt: <code>{AliasName}Locator.ts</code> (khai báo locator) và <code>{AliasName}Page.ts</code> (action methods) từ component-rule + userflow
Arguments	<code><group> <alias> / [alias1 alias2] / --all / --pbi <PBI_ID></code>

INPUT:

File	Bắt buộc	Ghi chú
<code>vnr-qckit/detected_components/component_temp_{alias}.md</code>	✓	Thiếu → DỪNG
<code>.specify/tests/url-aliases.md</code>	✓	Alias → path + group detection
<code>.specify/rules/component-rule.md</code>	Ưu tiên	Selector rules
<code>vnr-qckit/templates/template-component-rule.md</code>	Fallback	Dùng khi thiếu component-rule.md
<code>.specify/memory/userflow-{group}.md</code>	Ưu tiên	Tìm trước theo group
<code>.specify/memory/userflow.md</code>	Fallback	Dùng khi không có file theo group
<code>.specify/specs/{PBI_ID}/testcase.md</code>	Tùy chọn	Suy luận method cần ưu tiên (với <code>--pbi</code>)

OUTPUT:

File	Trạng thái	Ghi chú
<code>.specify/tests/pages/{group}/{AliasName}Locator.ts</code>	Tạo mới / ghi đè	Chỉ khai báo Locator — không có action method
<code>.specify/tests/pages/{group}/{alias}/{AliasName}Page.ts</code>	Tạo mới / ghi đè	Import Locator, kế thừa BasePage, action methods
<code>.specify/tests/knowledge/{group}/{alias}/locator-map.md</code>	Tạo mới / ghi đè	Tài liệu tham chiếu locator

Quy tắc đặt tên:

```
alias hrm_employee (group hrm)
  → Locator : .specify/tests/pages/hrm/HrmEmployeeLocator.ts    (class HrmEmployeeLocator)
  → Page    : .specify/tests/pages/hrm/hrm_employee/HrmEmployeePage.ts (class HrmEmployeePage)

alias obj_goal_personal (group obj)
```

```
→ Locator : .specify/tests/pages/obj/ObjGoalPersonalLocator.ts
→ Page    : .specify/tests/pages/obj/obj_goal_personal/ObjGoalPersonalPage.ts
```

Cấu trúc Locator file:

```
import { Page, Locator } from '@playwright/test'

export class HrmEmployeeLocator {
  // — Toolbar —————
  readonly btnThemMoi: Locator // Thêm mới – toolbar

  // — Form Fields – dialog —————
  readonly tenNhanVien: Locator // Tên nhân viên (k-textbox)
  readonly phongBan: Locator // Phòng ban (k-combobox)

  constructor(private readonly page: Page) {
    this.btnThemMoi = page.getByRole('button', { name: 'Thêm mới' })
    this.tenNhanVien = page
      .locator('div:has(> div.FieldTitle, > div.FieldValue)')
      .filter({ has: page.locator('div.FieldTitle label', { hasText: 'Tên nhân viên' }) })
  }

  .locator('div.FieldValue input.k-textbox')
  this.phongBan = page.locator('[formcontrolname="DepartmentId"] span.k-combobox
input.k-input')
}
}
```

Cấu trúc Page Object file:

```
import { Page } from '@playwright/test'
import { BasePage } from '../BasePage'
import { HrmEmployeeLocator } from '../HrmEmployeeLocator'

export class HrmEmployeePage extends BasePage {
  readonly url = '/employees'
  readonly loc: HrmEmployeeLocator

  constructor(page: Page) {
    super(page)
    this.loc = new HrmEmployeeLocator(page)
  }

  async goto() { await this.navigate(this.url) }
  async openCreateForm() { await this.loc.btnThemMoi.click() }
  async fillCreateForm(data: { tenNhanVien?: string, ... }) { ... }
  async submitForm() { await this.loc.btnLuu.click() }
  async createRecord(data) { await this.openCreateForm(); await
this.fillCreateForm(data); await this.submitForm() }
}
```

Ưu tiên locator strategy:

1. getByLabel('{label}') – label rõ ràng
 2. getByRole('{role}', { name: '{label}' }) – ARIA role
 3. locator(container).filter({ has: hasText(label) }).locator(input) – custom component
 4. locator('[formcontrolname="'....]') – formControlName
 5. locator(':nth-child(n) input') – fallback index
- [TODO]

Luồng xử lý:

BƯỚC 0 → Xác định alias list + group; kiểm tra file nguồn
 BƯỚC 1 → Đọc song song: url-aliases.md, component-rule.md, component_temp files, userflow files
 → Build ComponentRuleMap + FieldInventory + ActionMap
 BƯỚC 2 → Tính locator strategy tối ưu cho từng field
 BƯỚC 3 → Sinh Locator.ts + Page.ts (ghi đè nếu đã tồn tại)
 BƯỚC 4 → Sinh locator-map.md tài liệu tham chiếu
 BƯỚC 5 → Cập nhật _IMPACT_INDEX.json và flow-index.md nếu có

Skill: `qc_url_page_map`

Mục đích	Quét <code>*Page.ts</code> / <code>*Locator.ts</code> đã có, khớp với url-aliases.md, sinh mapping
Trigger	Chạy trước <code>qc_sub-system-map</code> , hoặc khi cần biết màn hình nào đã có Page Object

INPUT:

File	Bắt buộc	Ghi chú
<code>.specify/tests/url-aliases.md</code>	✓	Danh sách alias + path
<code>.specify/tests/pages/**/*Page.ts</code>	—	Các Page Object đã có
<code>.specify/tests/pages/**/*Locator.ts</code>	—	Các Locator đã có

OUTPUT:

File	Trạng thái	Ghi chú
<code>.specify/tests/pages/url-page-map.md</code>	Tạo mới / ghi đè	Bảng mapping alias → Page Object

Format url-page-map.md:

```
# URL → Page Object Map
> Tổng alias: 260 | Đã khớp: 18 | Chưa có PO: 242

## Đã có Page Object (18 màn hình)
| Alias | URL Path | Mô tả | Page Class | Page File | Locator Class | Locator File |

## Chưa có Page Object (242 màn hình)
| Alias | URL Path | Mô tả | Section |
```

Thống kê theo Section

| Section | Tổng | Đã có PO | Chưa có PO |

Thuật toán khớp (score ≥ 0.6):

```
alias att_leave_day → chuẩn hóa "attleaveday"  
Page class AttLeaveDayPage → chuẩn hóa "attleaveday"  
Score = số ký tự khớp liên tiếp / độ dài chuỗi ngắn hơn
```

Skill: `qc_sub-system-map`

Mục đích Trích xuất mapping màn hình → group từ url-page-map.md, sinh `sub-system-map.json`

Trigger Chạy sau `qc_url_page_map`; `qc_auto_test` đọc file này để lấy URL + Locator

INPUT:

File	Bắt buộc	Ghi chú
<code>.specify/tests/pages/url-page-map.md</code>	✓	Chỉ đọc section "Đã có Page Object"

OUTPUT:

File	Trạng thái	Ghi chú
<code>.specify/memory/sub-system-map.json</code>	Tạo mới / ghi đè	JSON mapping màn hình → group + Page Object path

Format sub-system-map.json:

```
{  
  "success": true,  
  "generated_at": "2026-06-11T...",  
  "total": 18,  
  "data": {  
    "rows": [  
      { "ma_man_hinh": "att_leave_day", "ten_man_hinh": "DS Ngày nghỉ", "group": "att"  
    }  
    ]  
  }  
}
```

Giai đoạn 4 — Sinh Test Case

Sinh testcase.md, datafake.json, và knowledge từ spec của PBI.

Skill: `qc_generate`

Mục đích	Sinh bộ automation test đầy đủ cho một PBI từ tài liệu spec
Arguments	<PBI_ID> (bắt buộc)

INPUT:

File	Bắt buộc	Ghi chú
.specify/specs/{PBI_ID}/spec.md	✓	Thiếu → DỪNG ngay
.specify/specs/{PBI_ID}/plan.md	✓	Thiếu → DỪNG ngay
.specify/memory/userflow-{group}.md	✓	Thiếu → DỪNG, hỏi group màn hình
.specify/templates/testcase_template.md	—	Template chuẩn — ưu tiên dùng nếu có
.specify/memory/constitution.md	—	Quy tắc dự án
.specify/memory/domain-knowledge.md	—	Business domain
.specify/specs/{PBI_ID}/assets/*.png, *.jpg	—	UI mockup (vision)
.specify/tests/data-catalog/categories/	—	Dữ liệu thực (vendors, customers, ...)

OUTPUT:

File	Trạng thái	Ghi chú
.specify/specs/{PBI_ID}/testcase.md	Tạo mới	Danh sách test case với Mục 4 (bảng trạng thái)
.specify/specs/{PBI_ID}/datafake.json	Tạo mới	Dữ liệu giả có nghĩa nghiệp vụ
.specify/tests/knowledge/{PBI_ID}/screen-summary.md	Tạo mới	Mô tả màn hình, luồng, business rules
.specify/tests/knowledge/{PBI_ID}/field-catalog.md	Tạo mới	Danh mục field và validation
.specify/tests/knowledge/_IMPACT_INDEX.json	Tạo mới / cập nhật	Luôn tạo/cập nhật — không cần tạo thủ công

Cấu trúc testcase.md (khi dùng template):

```
## 4. Danh sách Test Case
| TC | Tên | Loại | Ưu tiên | Trạng thái |
| --- | --- | --- | --- | --- |
| TC-001 | Hiển thị form đủ 3 field | ✓ Auto | --uutien | 🟡 Chưa test |
| TC-002 | Lưu thành công | ✓ Auto | p0 | 🟡 Chưa test |
| TC-005 | App mobile | 🖐 Manual | p2 | 🟡 Chưa test |
```

TC ID format:

- Khi có template: {PBI_ID}_{GROUP}_{FEATURE_CODE}_{NNN} (ví dụ: 15552_CAT_CHUYENKHO_001)
- Khi không có template: TC-NNN (fallback)

Khái niệm **--uutien**:

- TC thiết yếu nhất — nếu fail thì PBI không thể chấp nhận
- Mỗi PBI: **3–7 TC ưu tiên**, không quá 40% tổng TC
- `qc_auto_test` chạy **--uutien** trước; nếu fail → dừng toàn bộ batch

Phân loại Auto / Manual:

Loại	Điều kiện
 Auto	Tất cả steps qua Web browser, verify qua DOM, message lỗi đã xác định
 Manual	Cần thiết bị vật lý / visual check / message [TODO] / data setup phức tạp trong DB

Mặc định là Auto — chỉ chuyển Manual khi có lý do cụ thể. Mỗi TC phải có **Lý do loại**.

Luồng xử lý (6 Bước):

```
BƯỚC 1 → Đọc tất cả tài liệu song song
BƯỚC 2 → Phân tích spec: thông tin màn hình, trường dữ liệu, nhóm test; đánh dấu --uutien
BƯỚC 3 → Sinh testcase.md (dùng template nếu có; fallback format TC-NNN)
BƯỚC 4 → Sinh datafake.json (happy_path, validation, edge_cases)
BƯỚC 5 → Sinh knowledge/*.md (screen-summary.md, field-catalog.md)
BƯỚC 6 → Tạo/cập nhật _IMPACT_INDEX.json (luôn thực hiện)
```

Quy tắc data fake: Không dùng `foo`, `bar`, `test123`. Ưu tiên data-catalog → domain-knowledge → sinh theo format VN.

Giai đoạn 5 — Chạy Test Tự động

Skill: `qc_auto_test`

Mục đích	Đọc testcase.md, bóc tách bước → Page Object/Locator, sinh spec.ts có cấu trúc, thực thi, cập nhật kết quả vào Mục 4, cập nhật Locator/Page khi phát hiện component mới
Engine	playwright-cli (tất cả execution: navigate, fill, click, assert, trace) + playwright-mcp (chỉ discovery DOM/component mới và Kendo combobox)
Arguments	<PBI_ID> (bắt buộc) / --reset TC-001,TC-002 / --reset-all

Nguyên tắc tool: `playwright-cli` = mọi thao tác thực thi. `playwright-mcp` = **chỉ** khi cần đọc DOM để discovery selector mới hoặc xử lý Kendo combobox popup động.

INPUT (đọc từng file đúng lúc cần, không gom song song):

File	Bắt buộc	Ghi chú
<code>.specify/tests/.env + .env.{ENV}</code>		Đọc đầu tiên — lấy APP_URL
<code>.specify/specs/{PBI_ID}/testcase.md</code>		Đọc thứ 2 — lọc TC, cập nhật cột 

File	Bắt buộc	Ghi chú
<code>.specify/memory/sub-system-map.json</code>	✓	Đọc thứ 3 — lấy URL + Locator/Page path
<code>{AliasName}Locator.ts</code> + <code>{AliasName}Page.ts</code>	✓	Đọc per-màn hình — bóc tách selector + method
<code>.specify/tests/user.json</code>	✓	Đọc trước login check
<code>.specify/specs/{PBI_ID}/datafake.json</code>	✓	Đọc trước khi sinh spec.ts

OUTPUT:

File	Trạng thái	Ghi chú
<code>.specify/specs/{PBI_ID}/testcase.md</code>	Cập nhật	Cột Trạng thái + cột ★ trong Mục 4
<code>.specify/specs/{PBI_ID}/auto_test_results/specs/{PBI_ID}.spec.ts</code>	Tạo mới	Spec có cấu trúc (import Locator/Page, describe/test blocks)
<code>.specify/specs/{PBI_ID}/auto_test_results/traces/TC-{NNN}.zip</code>	Tạo mới	Trace per-TC
<code>.specify/specs/{PBI_ID}/auto_test_results/screenshots/TC-{NNN}-fail-*.png</code>	Tạo khi FAILED	Ảnh chụp khi assertion thất bại / auth expired
<code>.specify/specs/{PBI_ID}/auto_test_results/run-{DATETIME}-summary.md</code>	Tạo mới	Tóm tắt lần chạy (có section "Component mới phát hiện")
<code>.specify/specs/{PBI_ID}/auto_test_results/HISTORY.md</code>	Append	Engine: <code>cli+mcp</code> ; cột New components
<code>.specify/memory/components/component_{alias}.md</code>	Cập nhật nếu phát hiện component mới	Thêm field mới có tag <code>[NEW - /qc_auto_test {PBI_ID}]</code>
<code>{AliasName}Locator.ts</code>	Cập nhật nếu phát hiện selector mới	Thêm constant mới

File	Trạng thái	Ghi chú
<code>{AliasName}Page.ts</code>	Cập nhật nếu cần method mới	Thêm action method

Phân công tool:

Nhiệm vụ	Tool
Tất cả execution (navigate, fill, click, check, assert, trace, screenshot)	<code>playwright-cli</code>
Load/save session	<code>playwright-cli load_storage_state / save_storage_state</code>
Discovery selector mới (chưa có trong Locator.ts)	<code>browser_snapshot</code> (mcp)
Auth guard — kiểm tra redirect login	<code>browser_current_url</code> (mcp)
Kendo combobox popup động	<code>browser_click</code> + <code>browser_type</code> + <code>browser_snapshot</code> + <code>browser_click</code> (mcp)
Chụp ảnh tức thì khi lỗi	<code>browser_screenshot</code> (mcp)

⚠️ Không dùng `browser_navigate` cho execution — chỉ `playwright-cli navigate`.

Thứ tự ưu tiên chạy:

```
1. --uutien  (FAILED → dừng toàn bộ batch)
2. p0
3. p1
4. p2
5. p3+ / không nhận
```

Trạng thái kết quả:

Trạng thái	Điều kiện	Lần sau
✅ PASSED	Tất cả assertion pass	Không chạy lại
❌ FAILED	Ít nhất 1 assertion thất bại	Không chạy lại
⏸ SKIPPED	Precondition không thỏa / [TODO] assertion	Chạy lại
⏸ NOT_RUN	Dừng do --uutien FAILED / AUTH_EXPIRED	Chạy lại
🟡 Chưa test	Chưa từng chạy	Chạy lại

Cấu trúc spec.ts sinh ra:

```
// {PBI_ID}.spec.ts – sinh bởi /qc_auto_test
import { CatLeaveDayTypeLocator } from '../..pages/cat/CatLeaveDayTypeLocator'
```

```
import { CatLeaveDayTypePage } from
'../../pages/cat/cat_leave_day_type/CatLeaveDayTypePage'
import datafake from '../../specs/{PBI_ID}/datafake.json'

// Helper functions: waitForToast(), checkAuthGuard()

test.describe('★ --uutien', () => {
  test.beforeEach(async ({ page }) => { await checkAuthGuard(page) })

  test('[TC-001] @{PBI_ID} @auto @uutien – Tên TC', async ({ page }) => {
    const screenPage = new CatLeaveDayTypePage(page)
    await screenPage.goto()
    await page.fill(CatLeaveDayTypeLocator.FORM_TEN_LOAI, datafake.happy_path.tenLoai)
    await screenPage.submitForm()
    await waitForToast(page, 'Lưu thành công')
  })
})

test.describe('p0', () => { ... })
test.describe('p1', () => { ... })
```

Luồng xử lý (7 Bước):

BƯỚC 0 → Đọc .env → testcase.md (lọc TC, sắp xếp) → sub-system-map.json
 Đọc Locator.ts + Page.ts per-màn hình → bóc tách selector + method
 Đánh dấu [MISSING_SELECTOR] cho field chưa có trong Locator.ts
 Cập nhật cột ★ trong Mục 4 testcase.md (thêm cột nếu thiếu)

BƯỚC 1 → Đọc user.json → playwright-cli load_storage_state → login check
 PASSED → playwright-cli save_storage_state ngay

BƯỚC 2 → Đọc datafake.json → sinh specs/{PBI_ID}.spec.ts có cấu trúc:
 - Import Locator/Page từ sub-system-map paths
 - Discovery [MISSING_SELECTOR] qua browser_snapshot (mcp)
 - Cập nhật component_{alias}.md + Locator.ts + Page.ts nếu có component mới

BƯỚC 3 → Thực thi từng TC tuần tự (playwright-cli chính):
 A. playwright-cli tracing_start per-TC
 B. Auth guard: browser_current_url (mcp)
 C. Thực thi bước: playwright-cli navigate/fill/click/check
 Kendo combobox: mcp sequence (click→type→snapshot→click)
 D. Assertion: playwright-cli assert_visible/text/url/count/value
 E. playwright-cli tracing_stop → lưu traces/TC-{NNN}.zip
 F. Cập nhật Mục 4 ngay → kiểm tra điều kiện dừng

BƯỚC 4 → Xác định trạng thái cuối từng TC → cập nhật Mục 4

BƯỚC 5 → Ghi run-{DATETIME}-summary.md (có section "Component mới phát hiện")

BƯỚC 6 → Append HISTORY.md (cột Engine: cli+mcp; cột New components)

BƯỚC 7 → Cập nhật bảng Tóm tắt trong testcase.md

Quy tắc quan trọng:

- playwright-cli = tất cả execution; playwright-mcp = discovery only
- **Đọc file từng cái đúng lúc cần** — không gom song song ở đầu
- **Selector dùng constant** từ Locator.ts (SNAKE_CASE ALL_CAPS), không hard-code trong test body
- **Data dùng datafake.json** — không hard-code giá trị trong test
- TC  PASSED /  FAILED → không chạy lại; TC  SKIPPED /  Chưa test → chạy lại

- **Login check bắt buộc** → PASSED phải `save_storage_state` ngay; FAILED → dừng toàn bộ
- **TC --uutien FAILED** → dừng toàn bộ batch
- *Không tự động gợi ý chạy /qc_ khác* khi gặp lỗi — chỉ thông báo rõ ràng*

Giai đoạn 6 — Phân tích & Triage kết quả

Skill: `qc_triage`

Mục đích	Đọc kết quả run mới nhất từ HISTORY.md, phân loại từng TC thành PASS / APP_BUG / INFRA, xác định Severity cho APP_BUG
----------	---

Arguments `<PBI_ID>` (bắt buộc)

INPUT:

File	Bắt buộc	Ghi chú
<code>.specify/specs/{PBI_ID}/auto_test_results/HISTORY.md</code>		Thiếu → DỪNG (chạy qc_auto_test trước)
<code>.specify/specs/{PBI_ID}/auto_test_results/{SUMMARY_FILE}</code>	Ưu tiên	Summary file của run mới nhất
<code>.specify/specs/{PBI_ID}/testcase.md</code>		Luôn đọc để lấy mô tả, nhóm, priority

OUTPUT:

File	Trạng thái	Ghi chú
<code>.specify/specs/{PBI_ID}/auto_test_results/triage-{PBI_ID}-{YYYYMMDD}.md</code>	Tạo mới / ghi đè	Báo cáo triage đầy đủ

3 loại phân loại:

Loại	Dấu hiệu
 PASS	Tất cả assertion green, không timeout, không selector error
 APP_BUG	Script đúng, infra ổn — app hoạt động sai nghiệp vụ (feature missing, validation thiếu, business rule sai, DB không persist, filter sai)
 INFRA	Lỗi không phải do code app: session expired, host mismatch, network timeout, stale selector, seed data thiếu, config sai, script bug

Severity cho APP_BUG:

Severity	Định nghĩa
 Critical	Feature core bị vỡ hoàn toàn / security violation / data loss
 High	Business rule P1 sai (block sai / pass sai) / validation bắt buộc thiếu

Severity**Định nghĩa**

● Medium	Message sai / field không persist / conditional show/hide sai
● Low	UI cosmetic, label sai, default value sai (không ảnh hưởng save)

Format triage-{PBI_ID}-{DATE}.md:

```
# Triage Report – {PBI_ID}
> Run: {RUN_ID} – {DATETIME} | Triage: {ISO_DATE}

## Tổng quan
| Phân Loại | Số TC | % |
| ✓ PASS | n | % |
| 🐛 APP_BUG | n | % |
| 🛠 INFRA | n | % |

## 🐛 APP_BUG – Bảng Severity
| Severity | TC | Tên | Mô tả Lỗi | Action |

## 🛠 INFRA
| TC | Nguyên nhân | Loại INFRA | Action fix |

## Hành động tiếp theo (ưu tiên)
### Ngay (block test): Fix INFRA trước
### Sau INFRA: Fix APP_BUG Critical/High
```

Luồng xử lý (5 Bước):

BƯỚC 1 → Kiểm tra HISTORY.md, xác định run mới nhất, đọc summary + testcase.md
 BƯỚC 2 → Phân loại từng TC (PASS / APP_BUG / INFRA) theo bảng quyết định
 BƯỚC 3 → Xác định Severity cho mỗi APP_BUG
 BƯỚC 4 → Sinh file triage-{PBI_ID}-{YYYYMMDD}.md
 BƯỚC 5 → Báo cáo console (KHÔNG sửa testcase.md hay spec.ts)

Quy tắc: Ưu tiên phân loại INFRA trước APP_BUG — INFRA có thể che giấu APP_BUG thực sự. Nếu cùng 1 TC có cả dấu hiệu INFRA + APP_BUG → phân loại INFRA + ghi chú "Cần xác nhận sau khi fix INFRA".

Sơ đồ luồng đầy đủ (theo thứ tự sử dụng)**GIAI ĐOẠN 1 – CÀI ĐẶT (chạy 1 lần)**

```
/qc_pre
├─ /qc_setup (tuần tự trước)
│   ├── npm install @playwright/test
│   ├── copy Chromium → %LOCALAPPDATA%
│   ├── tạo playwright.config.ts
│   └─ tạo global-setup.ts
└─ Song song:
    └─ /qc_basepage → BasePage.ts
```

└─ crawl navigation → url-aliases.md



GIAI ĐOẠN 2 – KHÁM PHÁ ỨNG DỤNG

```

/qc_detect_component
├─ navigate + evaluate qua playwright-mcp
├─ sinh component_{alias}.md
│   └─ .specify/memory/components/

/qc_component_rule (sau detect_component)
├─ tổng hợp component types mới
├─ sinh/cập nhật component-rule.md
│   └─ .specify/rules/

/qc_user_flow
├─ crawl action qua playwright-mcp
├─ vẽ ASCII flow diagram 5 tầng
├─ sinh userflow-{group}.md
│   └─ .specify/memory/
└─ cập nhật flow-index.md
  
```



GIAI ĐOẠN 3 – MAPPING PAGE OBJECT

```

/qc_map_flow
├─ đọc component-rule.md + userflow-{group}.md
├─ sinh {AliasName}Locator.ts
│   └─ pages/{group}/
├─ sinh {AliasName}Page.ts
│   └─ pages/{group}/{alias}/
├─ sinh locator-map.md
│   └─ knowledge/{group}/{alias}/

/qc_url_page_map
├─ quét *Page.ts + *Locator.ts
└─ sinh url-page-map.md

/qc_sub-system-map
├─ sinh sub-system-map.json
│   └─ .specify/memory/
  
```



GIAI ĐOẠN 4 – SINH TEST CASE (per PBI)

```

/qc_generate <PBI_ID>
├─ [BẮT BUỘC] spec.md + plan.md + userflow
├─ [tùy chọn] testcase_template.md
├─ sinh testcase.md (Mục 4 + --uutien)
├─ sinh datafake.json
├─ sinh knowledge/*.md
└─ tạo/cập nhật _IMPACT_INDEX.json
  
```





Bảng tóm tắt Input/Output nhanh

Skill	Input chính	Output chính	Ghi đề?
qc_setup	.env, plugin tools	playwright.config.ts, global-setup.ts, tsconfig.json	Tạo/sửa
qc_basepage	template hardcode	BasePage.ts	Tạo nếu thiếu
qc_pre	.env, app đang chạy	url-aliases.md	Cập nhật
qc_detect_component	url-aliases.md, component-rule.md, app (mcp)	component_{alias}.md → .specify/memory/components/	Ghi đề
qc_component_rule	component_*.md → .specify/memory/components/	component-rule.md	Append/Update
qc_user_flow	url-aliases.md, app (mcp)	userflow-{group}.md, flow-index.md	Cập nhật

Skill	Input chính	Output chính	Ghi đề?
qc_map_flow	component_temp_{alias}.md, component-rule.md, userflow- {group}.md	{AliasName}Locator.ts, {AliasName}Page.ts, locator- map.md	Ghi đề
qc_url_page_map	url-aliases.md, *Page.ts, *Locator.ts	url-page-map.md	Ghi đề
qc_sub-system-map	url-page-map.md	sub-system-map.json	Ghi đề
qc_generate	spec.md, plan.md (cả hai bắt buộc), userflow-{group}.md	testcase.md (Mục 4), datafake.json, knowledge/*.md, _IMPACT_INDEX.json	Tạo mới
qc_auto_test	testcase.md (Mục 4), datafake.json, sub-system- map.json (bắt buộc), Locator.ts/Page.ts	testcase.md (cập nhật Mục 4 + ★), specs/{PBI_ID}.spec.ts (structured), traces/, screenshots/, HISTORY.md; cập nhật component_{alias}.md + Locator.ts + Page.ts nếu phát hiện component mới	Cập nhật/Append
qc_triage	HISTORY.md, testcase.md	trriage-{PBI_ID}- {YYYYMMDD}.md	Ghi đề (cùng ngày)

Cấu trúc thư mục output tổng hợp

```

.specify/
├── tests/
│   ├── .env                ← ENV=local
│   ├── .env.local          ← APP_URL=...
│   ├── user.json           ← session state (sinh bởi global-setup.ts)
│   ├── playwright.config.ts ← sinh bởi qc_setup
│   ├── global-setup.ts     ← sinh bởi qc_setup
│   ├── tsconfig.json       ← sinh bởi qc_setup
│   ├── url-aliases.md      ← sinh bởi qc_pre (crawl navigation)
│   └── pages/
│       ├── BasePage.ts     ← sinh bởi qc_basepage
│       ├── url-page-map.md ← sinh bởi qc_url_page_map
│       └── {group}/
│           ├── {AliasName}Locator.ts ← sinh bởi qc_map_flow (locator only)
│           ├── {alias}/
│           └── {AliasName}Page.ts     ← sinh bởi qc_map_flow (actions + import
Locator)
├── tests/                  ← testDir cho Playwright
├── knowledge/
│   ├── _IMPACT_INDEX.json ← tạo/cập nhật bởi qc_generate
│   ├── {PBI_ID}/
│   │   ├── screen-summary.md ← sinh bởi qc_generate
│   │   └── field-catalog.md  ← sinh bởi qc_generate
│   └── {group}/{alias}/
│       └── locator-map.md    ← sinh bởi qc_map_flow

```

```

├─ specs/
│   └─ {PBI_ID}/
│       ├── spec.md           ← INPUT bắt buộc (trước qc_generate)
│       ├── plan.md          ← INPUT bắt buộc (trước qc_generate)
│       ├── assets/*.png     ← INPUT optional (mockup)
│       └── testcase.md      ← sinh bởi qc_generate, cập nhật Mục 4 bởi
qc_auto_test
│   ├── datafake.json        ← sinh bởi qc_generate
│   └─ auto_test_results/
│       └─ {PBI_ID}.spec.ts  ← structured spec (import Locator/Page,
describe/test blocks)
│   ├── specs/               ← thư mục chứa spec.ts
│   │   └─ {PBI_ID}.spec.ts
│   ├── traces/
│   │   └─ TC-{NNN}.zip      ← trace per-TC (network + DOM + screenshot)
│   ├── screenshots/
│   │   └─ TC-{NNN}-fail-*.png ← chụp khi FAILED / auth expired
│   ├── run-{DATETIME}-summary.md ← bao gồm "Component mới phát hiện"
│   ├── HISTORY.md           ← append (cột Engine: cli+mcp; cột New components)
│   └─ triage-{PBI_ID}-{YYYYMMDD}.md ← sinh bởi qc_triage
├─ memory/
│   ├── constitution.md      ← INPUT (quy tắc dự án)
│   ├── domain-knowledge.md  ← INPUT (business domain)
│   ├── flow-index.md        ← sinh/cập nhật incremental bởi qc_user_flow
│   └─ userflow-{group}.md   ← sinh bởi qc_user_flow (theo group: att, hre,
sal...)
│   ├── userflow-all.md      ← sinh bởi qc_user_flow --all
│   └─ sub-system-map.json    ← sinh bởi qc_sub-system-map
├─ rules/
│   └─ component-rule.md      ← sinh bởi qc_component_rule
├─ components/
│   └─ component_{alias}.md   ← sinh bởi qc_detect_component
└─ templates/
    └─ testcase_template.md   ← INPUT optional (qc_generate đọc nếu có)

```

Tài liệu này được tổng hợp từ đọc toàn bộ *SKILL.md* trong *vn-r-qckit* plugin. Cập nhật: 2026-06-11.